学习笔记

数据清洗和特征处理方法：

1. 选取特征：

Pclass, Sex, Age, SibSp, Parch, Fare

（这几个数据对目标产生直接影响；当然后期为了使预测更精确可以对 SibSp 和 Parch 两者进行处理构造新特征）

2. 对缺失值（本次所用标签中仅 Age 存在缺失）：

使用中位数填补 X_train["Age"] = X_train["Age"].fillna(X_train["Age"].median())----受极端值影响比平均值小

（假设有文字形式标签的缺失值可以用众数填补 or 用独热代码单独新增一列 none）

3. 对文字形式标签（sex）：

独热代码分为 male 和 female 两列 X = pd.get_dummies(X, dummy_na=False)

4. 数据标准化处理：

对所有数据进行标准化处理，避免不同特征的尺度差异大影响梯度下降优化

```
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std
```

注意：这里必须只能使用 x_train 里的 std 和 mean 对 test 集里的数据进行标准化，因为 test 里的数据理论上是未知的（这样才能真实测量模型的泛化能力）

模型结构、损失函数、优化器及主要超参数：

模型结构：

二分类逻辑回归模型

数学原理： X=[x1, x2, ... , xn]（列向量） W=[w1, w2, . . . , wn]

Y_hat=sigmoid(XW+b) 即 Y_hat=1/[1+e^(-z)]

对应结构：

输入层

|

| 7 个特征

↓

Linear 层

(Wx+b)

|

↓

Sigmoid 激活函数

|

↓

输出概率

代码表示：

#神经网络

net = nn.Sequential(nn.Linear(7,1),nn.Sigmoid())----------在 net 大层中新增共 7 个输入 1 个输出的 Linear 层和 Sigmoid 函数层

y_hat = net(X_train)----------将 7 个输入构成的矩阵输入到 net 大层中，得到的输出赋值给 y_hat

损失函数（本次使用二元交叉熵损失函数）：

数学理解：

当实际值 y=1 时，L= -log(y_hat)，即 y_hat 越接近 0，损失值越大；

当实际值 y=0 时，L= -log(1-y_hat)，即 y_hat 越接近 1，损失值越大.

为什么不用均方损失函数：

1）均方损失只是计算数值的距离远近，并未考虑概率分布；而二元交叉熵更符合最大似然估计

2）(y-y_hat)<1，均方损失梯度过小；而二元交叉熵损失梯度适中，满足里实际概率偏差越大损失越严重，变化平稳

代码表示：

loss_fn = nn.BCELoss() ----------定义损失函数为二元交叉熵损失

loss = loss_fn(y_hat,y_train.reshape(-1,1))----------计算损失

loss.backward()----------将 loss 向后返回梯度

优化器：梯度下降算法的执行者

optimizer=torch.optim.SGD(net.parameters(),lr=1)------------定义优化器 optimizer，采用 SGD(随机梯度下降算法)，训练参数 w，b（net.parameters()）,并且设置学习率为 1

optimizer.zero_grad()----------------清空优化器内参数梯度，避免梯度累加

optimizer.step()-----------使用 w，b 当前梯度利用最开始定义的优化方式更新 w，b

主要超参数：

学习率 lr（有意思的点是，因为该模型过于简单，lr=5 比 lr=0.1 训练出的 accuracy 效果还要好）

训练轮次 epoch（同样因为模型过于简单，训练个 100 次就够了）

数据划分方式、随机种子和实验配置：

数据划分：

分成了 80%的训练集和 20%的测试集（随机分配）

引入了 sklearn 库来进行划分：（好吧在最后提交的时候看到了还有 Dataset 方式，属实眼瞎）

```
X_train,X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

（询问 ai 后得出的结果，貌似 pytorch 里没有随机划分的函数）

*注意：为了划分的有效性，对划分号的 test 集进行处理时只能用 train 集里的标准差等等数据，从而防止数据泄露影响对模型真实泛化能力的判断

随机种子：

随机种子是决定随机路径、控制随机起点的方法，设置随机起点有助于后续随机效果的复现。

本次代码共用两处随机种子：1.用 sklearn 库随机分配 train，test 集时使用 random_state=42；

2.在优化器最开始随机产生 w，b 时在前面加上 torch.manual_seed(42) （程序没有使用 numpy 库故没有对其种子加以说明）

实验配置：

Python 3.9.25

numpy: 1.21.6

pandas: 1.4.4

matplotlib: 3.5.3

scikit-learn: 1.1.3

torch: 2.8.0+cpu

（在 python3.9 的环境下，别的库不能随便升级，需要强行锁定版本安装不然会产生版本冲突）

训练过程中遇到的问题及解决方法：

1）从 0 开始编写模型代码的时候的确有点缺乏架构感，不知道第一步干什么第二步干什么总之就是有点迷茫

>解决：仔细阅读了一轮关于线性回归的模型架构，并让 gpt 帮我用中文列出第一步要是实现哪些功能的代码、第二步要实现哪些功能的代码……做着做着就轻松起来了

2）（有点弱智）最开始受《动手学深度学习》的影响先入为主的认为应该采用交叉熵损失函数来解决（就是对各个各个数据的得分取 e 的指数后看占比得出各组分概率）；但真正到代码层面就发现其不合理性了（不可能计算出生存的分数和死亡的分数，而且这么做从简洁性的角度看着就不合理），后来询问 gpt 后仔细阅读题干才发现题干已经给了是二元交叉损失（bushi

3）对 pytorch.nn 里面的具体函数有点记不清：

>看 b 站视频+结合 gpt 大致了解了下神经网络和不同架构对应的语句，后续还是想着在实战中记忆（总不能让之后任务的学习笔记白写 bushi）

测试集准确率、实验结果分析和心得体会

测试集准确率：79.88%（模型太过简单没法再往上提升----不过最高可以提升至 80.2%，不过为了画图呈现好看就稍微把训练轮次调少，学习率加大了一点点）

结果分析：准确率后续还可再提高（构造 or 引入新特征/优化模型增加神经网络层数），但考虑到本次为 pytorch 架构的初尝试就没有过多深入的优化模型。
根据最后的 w 值 tensor([[-0.7943, -0.4190, -0.3852, -0.1127, 0.1729, 0.5978, -0.7055]]，发现目标与 Pclass，Fare，Parch 有着更密切的关系，后续构造新特征时可考虑深挖这三个数据

心得体会：

感觉对 ml 的学习可以从数学角度、代码实现角度、模型优化角度入手，分别使用矩阵公式理解、代码记忆实战、画神经网络图/理论学习的方式加以完整的学习。